

Transfer Learning & Batch Normalization in Neural Networks

Neural Networks Course Project Report

Owner:

Tanush Shoor

Email ID:

tanush.shoor09@gmail.com

Course:

Neural Networks & Computer Vision

Professor:

Priyansh Saxena

Abstract

This project investigates two important deep learning techniques: Transfer Learning and Batch Normalization. In the first task, a Cats vs Dogs image dataset containing 1000 images was used to compare a CNN trained from scratch, a pre-trained ResNet18 with a frozen backbone, and a fine-tuned ResNet18. In the second task, a deep neural network was trained on the Fashion-MNIST dataset with and without Batch Normalization.

The models were evaluated using training loss and classification accuracy. The results show that Transfer Learning significantly improves performance on small datasets, while Batch Normalization improves training stability and convergence. These experiments provide practical insight into important techniques used in modern neural network training.

1. Introduction

Neural Networks are machine learning models inspired by the structure of the human brain. They learn patterns from data and are widely used in image classification, speech recognition, and many other applications.

This project focuses on two important deep learning concepts: Transfer Learning and Batch Normalization. Transfer Learning enables a model trained on a large dataset to be reused for a related task with limited data, while Batch Normalization helps stabilize and accelerate the training process.

The objective of this project is to study the impact of these techniques through practical experiments. Transfer Learning is evaluated using a Cats vs Dogs image classification task, while Batch Normalization is analyzed using the Fashion-MNIST dataset. The performance of different models is compared using training loss and classification accuracy.

2. Task 1: Transfer Learning on a Small Dataset

2.1 Problem

Training a neural network from scratch on a small dataset often results in lower accuracy due to limited feature learning.

Transfer Learning addresses this issue by reusing knowledge learned from a large dataset and applying it to a new task.

The objective of this task is to compare:

1. CNN trained from scratch
2. Frozen ResNet18
3. Fine-Tuned ResNet18

The models were evaluated using training loss and classification accuracy.

2.2 Approach

Dataset: <https://www.kaggle.com/datasets/aleemaparakatta/cats-and-dogs-mini-dataset/data>

The Cats vs Dogs Mini Dataset was selected for this experiment.

Dataset statistics:

- Cat images: 500
- Dog images: 500
- Total images: 1000

The dataset contains variations in animal pose, lighting conditions, backgrounds, and image quality, making it suitable for image classification experiments.

The dataset was divided into:

- Training Set: 80%
- Validation Set: 20%

Image Preprocessing

Before training, all images were:

- Resized to 224×224 pixels
- Converted to tensors
- Normalized when using ResNet18

These preprocessing steps ensure compatibility with the pre-trained network architecture and provide consistent input dimensions.

Model 1: CNN Trained from Scratch

A simple Convolutional Neural Network (CNN) was implemented and trained entirely from random initialization.

Architecture:

- Input Image
 - Convolution Layer (16 Filters)
 - ReLU Activation
 - Max Pooling
 - Convolution Layer (32 Filters)
 - ReLU Activation
 - Max Pooling
 - Convolution Layer (64 Filters)
 - ReLU Activation
 - Max Pooling
 - Fully Connected Layer
 - Output Layer (2 Classes)

The convolution layers extract image features while the fully connected layers perform the final classification.

Model 2: Frozen ResNet18

ResNet18 is a deep convolutional neural network originally trained on the ImageNet dataset containing millions of labeled images.

For this experiment:

- All convolutional layers were frozen
- Only the final classification layer was trained
- Pre-trained ImageNet features were reused

This allows the network to use previously learned visual features while adapting only the final classifier to the Cats vs Dogs dataset.

Model 3: Fine-Tuned ResNet18

In the fine-tuning approach:

- ResNet18 was initialized with ImageNet weights
- The final convolutional block was unfrozen
- The final classification layer was replaced and trained

This enables the network to adapt its learned features specifically to the cats and dogs classification task.



Figure 1. Sample images from the Cats vs Dogs Mini Dataset used for Transfer Learning experiments.

2.3 Experiments

Three controlled experiments were conducted.

Experiment 1: CNN From Scratch

The CNN model was trained from randomly initialized weights.

Training Loss:

Epoch	Loss
1	0.7968
2	0.6872
3	0.6451
4	0.6260
5	0.5838

Validation Accuracy: 61.5%

Experiment 2: Frozen ResNet18

The pre-trained ResNet18 model was used with all backbone layers frozen.

Training Loss:

Epoch	Loss
1	0.4628
2	0.2318
3	0.1501
4	0.1454
5	0.1238

Validation Accuracy: 97%

Experiment 3: Fine-Tuned ResNet18

The final convolutional block and classifier were trained while retaining pre-trained ImageNet features.

Training Loss:

Epoch	Loss
1	0.1743
2	0.0346
3	0.0087
4	0.0062
5	0.0038

Validation Accuracy: 99.5%

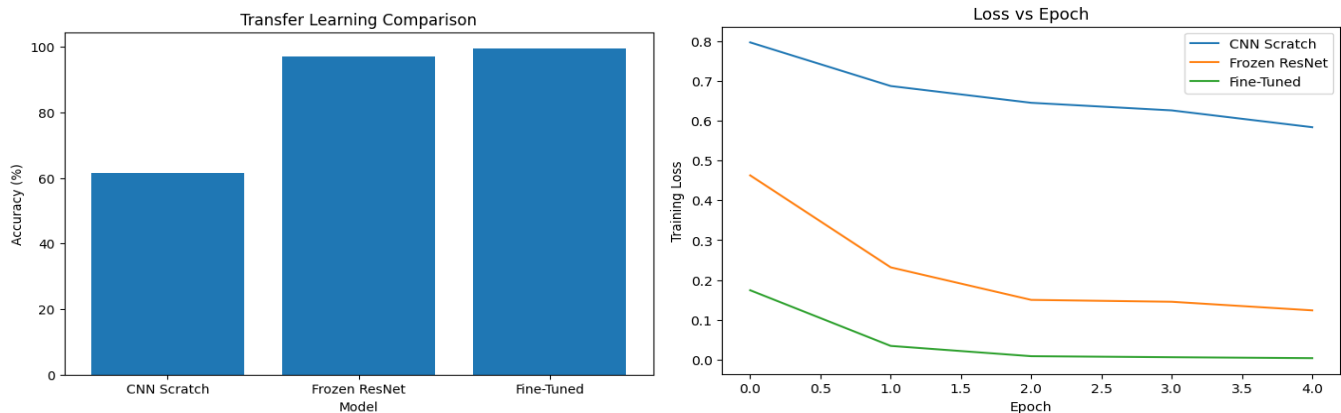


Figure 2. Accuracy comparison of the three Transfer Learning approaches.

Figure 3. Training loss comparison for CNN, Frozen ResNet18, and Fine-Tuned ResNet18.

2.4 Analysis

The CNN trained from scratch achieved the lowest accuracy (61.5%) because the small dataset provided limited data for learning useful image features.

The Frozen ResNet18 model achieved a much higher accuracy of 97%. Since it was pre-trained on ImageNet, it already contained useful visual feature representations and required only the final classification layer to be trained.

The Fine-Tuned ResNet18 model achieved the highest accuracy of 99.5%. Allowing the final layers to adapt to the cats and dogs dataset further improved performance.

Overall, the results show that Transfer Learning is highly effective for small datasets, significantly improving classification accuracy compared to training a CNN from scratch.

3. Task 2: Batch Normalization vs No Batch Normalization

3.1 Problem

Training deep neural networks can be difficult because changing activation distributions may slow learning and reduce training stability.

Batch Normalization addresses this issue by normalizing activations during training, helping networks converge faster and train more efficiently.

The objective of this task is to compare two identical deep neural networks:

1. Without Batch Normalization
2. With Batch Normalization

The models are evaluated using training loss, convergence behavior, and classification accuracy.

3.2 Approach

Dataset

The Fashion-MNIST dataset was used for this experiment.

Fashion-MNIST is a benchmark image classification dataset consisting of grayscale images belonging to ten different clothing categories.

Dataset statistics:

- Training Images: 60,000

- Test Images: 10,000
- Number of Classes: 10

The ten classes are:

1. T-shirt/Top
2. Trouser
3. Pullover
4. Dress
5. Coat
6. Sandal
7. Shirt
8. Sneaker
9. Bag
10. Ankle Boot

Each image has dimensions:

28×28 pixels

The dataset was automatically downloaded using the PyTorch dataset library.

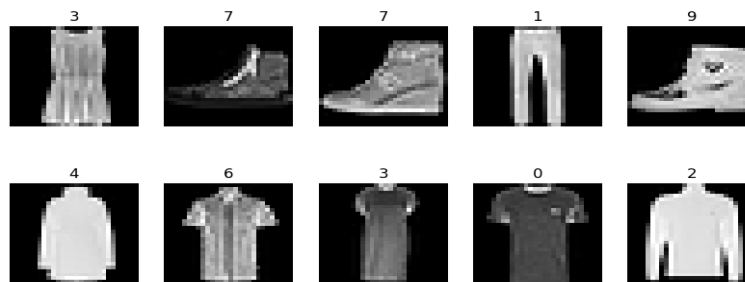


Figure 4. Sample images from the Fashion-MNIST dataset used for Batch Normalization experiments.

Neural Network Architecture

A fully connected deep neural network was implemented.

Architecture:

Input Layer (784 neurons)

→ Hidden Layer 1 (512 neurons)

→ Hidden Layer 2 (256 neurons)

→ Hidden Layer 3 (128 neurons)

→ Hidden Layer 4 (64 neurons)

→ Hidden Layer 5 (32 neurons)

→ Output Layer (10 neurons)

ReLU activation was applied after each hidden layer.

The network contains more than six trainable layers, satisfying the project requirement.

Model A: Without Batch Normalization

The first model consisted only of:

Linear Layer

→ ReLU

→ Linear Layer

→ ReLU

→ ...

→ Output Layer

No normalization layers were used.

Model B: With Batch Normalization

The second model used the exact same architecture, but BatchNorm1d layers were inserted after every hidden layer.

Architecture:

Linear Layer

→ BatchNorm

→ ReLU

→ Linear Layer

→ BatchNorm

→ ReLU

→ ...

→ Output Layer

This creates a controlled experiment because the only difference between the two models is the use of Batch Normalization.

What is Batch Normalization?

Batch Normalization normalizes activations within each mini-batch during training, helping stabilize learning and improve gradient flow through the network.

Its main benefits include:

- Faster convergence
- Improved training stability
- Easier optimization of deep networks

Although it may not always significantly increase final accuracy, it generally improves the overall training process.

3.3 Experiments

Three experiments were conducted.

Experiment 1: No Batch Normalization (10 Epochs)

Training Loss:

Epoch	Loss
1	0.6164
2	0.3994
3	0.3517
4	0.3273
5	0.3047
6	0.2866
7	0.2751
8	0.2636
9	0.2512
10	0.2402

Final Accuracy: 88.75%

Experiment 2: With Batch Normalization (10 Epochs)

Training Loss:

Epoch	Value
1	0.5523
2	0.3682
3	0.3277
4	0.2986
5	0.2810
6	0.2638
7	0.2447
8	0.2325
9	0.2218
10	0.2091

Final Accuracy: 89.57%

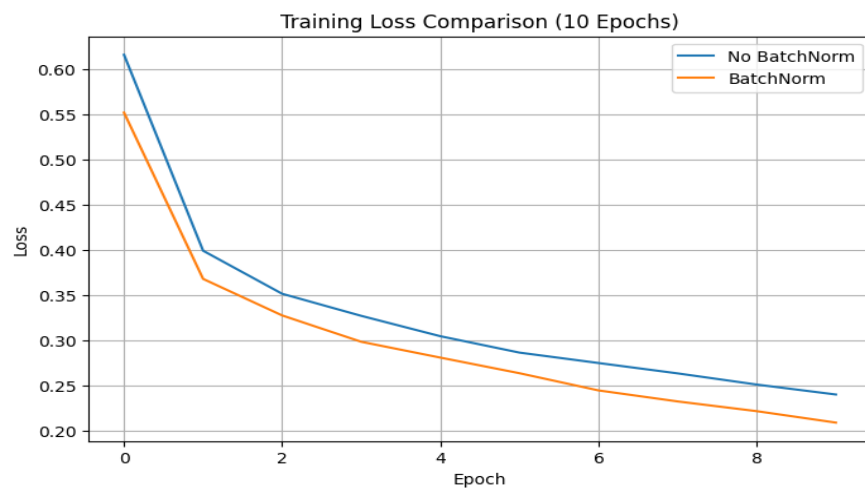


Figure 5. Training loss comparison between models with and without Batch Normalization over 10 epochs.

Experiment 3: Extended Training (20 Epochs)

To further study convergence behavior, both models were trained for 20 epochs.

Results:

Model	Accuracy
No BatchNorm	88.75%
BatchNorm	89.57%

Final Training Loss:

Model	Loss
No BatchNorm	0.1733
BatchNorm	0.1199

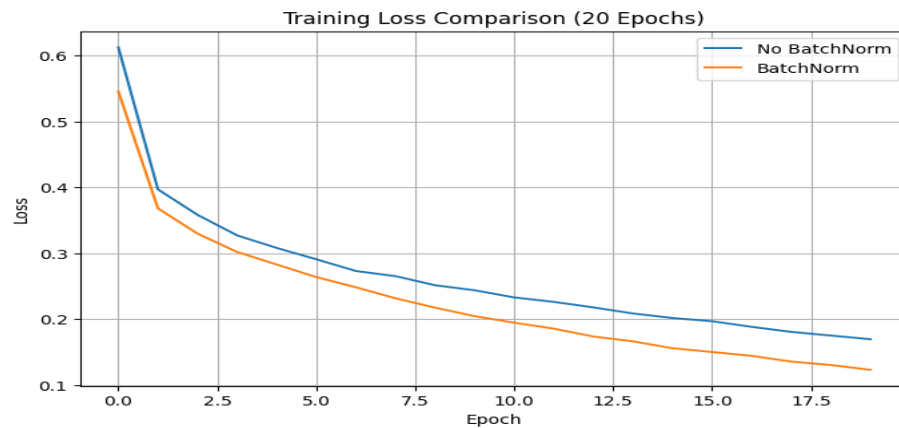


Figure 6. Training loss comparison between models with and without Batch Normalization over 20 epochs.

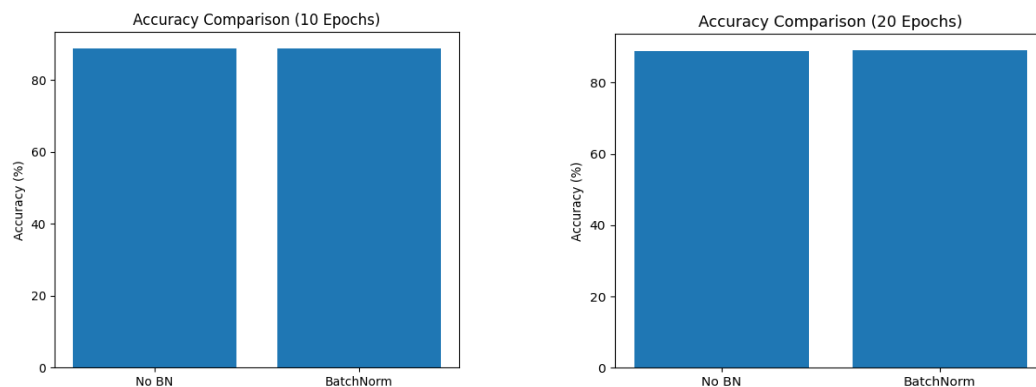


Figure 7. Accuracy comparison across all Batch Normalization experiments.

3.4 Analysis

The model with Batch Normalization achieved slightly higher accuracies than the model without Batch Normalization after both 10 and 20 epochs.

While the accuracy improvement was small, the training loss remained consistently lower with Batch Normalization. After 20 epochs, the loss decreased from 0.1733 (No BatchNorm) to 0.1199 (BatchNorm).

The loss curves also showed smoother convergence, indicating more stable training. Overall, the results suggest that Batch Normalization mainly improves optimization efficiency and training stability rather than significantly increasing final accuracy.

4. Conclusion

This project explored two important deep learning techniques: Transfer Learning and Batch Normalization.

In Task 1, Transfer Learning significantly improved performance on a small Cats vs Dogs dataset. The Fine-Tuned ResNet18 achieved the highest accuracy of 99.5%, demonstrating the advantage of using pre-trained models.

In Task 2, Batch Normalization resulted in lower training loss and slightly better accuracy compared to the model without Batch Normalization. It also produced more stable convergence during training.

Overall, the experiments showed that Transfer Learning improves performance on limited data, while Batch Normalization helps make neural network training more efficient and stable.